

Infrastructure Orchestration & Automation

Bloc 3 — Implémenter et automatiser le déploiement des solutions de données

TP guidé — Jour 2

Containeriser avec Docker, déployer avec Kubernetes

Enseignant	Dr. Abdelhak TOUITI
Durée	3h30 (après-midi)
Modalité	Individuel ou binôme
Compétences visées	C7.1, C8.2
Outils principaux	Docker, Docker Compose, Kubernetes (Minikube)

Fait suite au cours du matin : containerisation et ordonnancement.

Présentation du TP

Vous allez containeriser un service data simple (une API accompagnée d'une base de données), le faire fonctionner localement avec Docker Compose, puis le déployer sur un cluster Kubernetes local pour observer sa capacité à monter en charge et à s'auto-réparer.

Objectifs pédagogiques

- Écrire un Dockerfile pour containeriser une application.
- Faire communiquer deux conteneurs (API + base de données) avec Docker Compose et un volume persistant.
- Déployer ce service sur un cluster Kubernetes local (Minikube).
- Observer la résilience (auto-réparation) et la montée en charge (scaling) du déploiement.

Prérequis

- Docker et Docker Compose installés et fonctionnels.
- Python 3 disponible sur le poste (pour l'exemple d'API fourni).
- Un dépôt Git initialisé pour versionner vos fichiers.
- kubectl et Minikube : pas besoin de les avoir déjà installés, l'Étape 5 vous guide.

Livrable attendu

- Le dépôt Git contenant : Dockerfile, docker-compose.yml, deployment.yaml, service.yaml.
- Une capture d'écran de kubectl get pods montrant 3 réplicas Running.
- Une capture d'écran illustrant l'auto-réparation après suppression d'un pod.
- Les réponses aux questions de compréhension (section 9).

1 Préparer une API minimale

Étape 1

Créez un dossier tp-docker-k8s/api/ et placez-y les trois fichiers suivants.

1.1 — app.py

```
import os
from flask import Flask, jsonify
import psycopg2

app = Flask(__name__)

def connexion_db():
    return psycopg2.connect(
        host=os.environ.get("DB_HOST", "db"),
        dbname=os.environ.get("DB_NAME", "ventes"),
        user=os.environ.get("DB_USER", "postgres"),
        password=os.environ.get("DB_PASSWORD", "postgres"),
    )
```

```
@app.route("/health")
def health():
    return jsonify(status="ok")

@app.route("/ventes")
def ventes():
    conn = connexion_db()
    cur = conn.cursor()
    cur.execute("SELECT id, produit, montant FROM ventes;")
    rows = cur.fetchall()
    conn.close()
    return jsonify(rows)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

1.2 — requirements.txt

```
flask==3.0.3
psycpg2-binary==2.9.9
```

1.3 — init.sql (crée une table de démonstration)

```
CREATE TABLE ventes (
    id SERIAL PRIMARY KEY,
    produit VARCHAR(50),
    montant NUMERIC
);
INSERT INTO ventes (produit, montant) VALUES
    ('Abonnement A', 120.00),
    ('Abonnement B', 89.90);
```

2 Écrire le Dockerfile

Étape 2

Toujours dans api/, créez le fichier Dockerfile :

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
CMD ["python", "app.py"]
```

Construisez l'image :

```
cd tp-docker-k8s/api
docker build -t mon-service:1.0 .
```

Vérification

`docker images` doit lister `mon-service` avec le tag `1.0`.

3 Assembler API et base de données avec Docker Compose

Étape 3

À la racine de `tp-docker-k8s/`, créez `docker-compose.yml` :

```
services:
  api:
    build: ./api
    ports:
      - "8000:8000"
    environment:
      DB_HOST: db
      DB_NAME: ventes
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: ventes
      POSTGRES_PASSWORD: postgres
    volumes:
      - donnees_db:/var/lib/postgresql/data
      - ./api/init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  donnees_db:
```

Démarrez l'ensemble :

```
docker compose up -d
```

Vérification

`Invoke-WebRequest http://localhost:8000/health` doit répondre `status : ok`, et `Invoke-WebRequest http://localhost:8000/ventes` doit renvoyer les deux lignes insérées par `init.sql`.

Astuce

En PowerShell, `curl` est un alias vers `Invoke-WebRequest`, qui renvoie un objet réponse brut plutôt que le JSON déjà interprété. Pour une API JSON, préférez `Invoke-RestMethod`, qui affiche directement le contenu désérialisé.

4 Vérifier la persistance des données

Étape 4

```
docker compose down
docker compose up -d
Invoke-RestMethod http://localhost:8000/ventes
```

Vérification

Les données doivent toujours être présentes après l'arrêt et le redémarrage : elles sont stockées dans le volume `donnees_db`, indépendamment du cycle de vie des conteneurs.

Astuce

Supprimez le volume avec `docker compose down -v` pour observer, à l'inverse, la perte des données : c'est la preuve que c'est bien le volume qui assure la persistance, pas le conteneur.

5 Installer et démarrer un cluster Kubernetes local

Étape 5

La classe travaille avec **Minikube** (un cluster Kubernetes à un seul nœud, exécuté localement) plutôt que Kind ou un cloud managé — c'est l'outil le plus simple à installer et à piloter pour découvrir Kubernetes.

5.1 — Installer Minikube et kubectl

```
winget install --id Kubernetes.minikube
winget install -e --id Kubernetes.kubectl
```

Astuce

Si `winget` n'est pas reconnu, ouvrez PowerShell en tant qu'administrateur, ou installez "App Installer" depuis le Microsoft Store (`winget` en fait partie sur les postes Windows 10/11 à jour). Après l'installation, **fermez et rouvrez votre terminal** pour que les deux commandes soient reconnues.

5.2 — Vérifier les installations

```
minikube version
kubectl version --client
```

5.3 — Démarrer le cluster

```
minikube start --driver=docker
kubectl get nodes
```

Astuce

`--driver=docker` demande explicitement à Minikube de faire tourner son cluster à l'intérieur d'un conteneur Docker, en s'appuyant sur le Docker Desktop déjà installé ce matin — pas besoin d'activer Hyper-V ni d'installer VirtualBox. Le tout premier démarrage télécharge l'image du cluster et peut prendre plusieurs minutes.

Vérification

`kubectl get nodes` doit afficher un node au statut `Ready`. Vous pouvez aussi lancer `minikube dashboard` pour ouvrir une interface graphique du cluster dans votre navigateur — pratique pour visualiser ce que les commandes `kubectl` des étapes suivantes produisent.

Astuce

Si `minikube start` échoue en indiquant que Docker n'est pas accessible, vérifiez que Docker Desktop est bien lancé (icône dans la barre des tâches) avant de relancer la commande.

6 Déployer l'API sur Kubernetes

Étape 6

6.1 — Rendre l'image disponible au cluster

```
minikube image load mon-service:1.0
```

6.2 — deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mon-service
  template:
    metadata:
      labels:
        app: mon-service
    spec:
      containers:
        - name: api
          image: mon-service:1.0
          ports:
            - containerPort: 8000
```

```
env:
  - name: DB_HOST
    value: db
```

6.3 — service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: api-service
spec:
  selector:
    app: mon-service
  ports:
    - port: 8000
      targetPort: 8000
  type: NodePort
```

6.4 — Appliquer les manifests

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Astuce

Pour cette étape, la base de données n'est pas redéployée sur Kubernetes : gardez-la sur Docker Compose et concentrez-vous sur le comportement du Deployment. La connexion complète API + base sur Kubernetes est proposée en bonus (section 10).

Vérification

`kubectl get pods` doit afficher 3 pods au statut Running. `kubectl get svc` doit lister `api-service`.

7 Observer la résilience et la montée en charge

Étape 7

7.1 — Auto-réparation

```
kubectl get pods
kubectl delete pod <nom-d-un-pod>
kubectl get pods
```

7.2 — Montée en charge

```
kubectl scale deployment api-deployment --replicas=5
kubectl get pods
```

Vérification

Après suppression, un nouveau pod doit apparaître automatiquement pour revenir à 3 réplicas. Après le `scale`, 5 pods doivent être Running.

8 Versionner le projet

Étape 8

```
cd tp-docker-k8s
git init
git add Dockerfile docker-compose.yml deployment.yml service.yml api/
git commit -m "TP Jour 2 : containerisation et déploiement Kubernetes"
```

Livrable attendu

Un dépôt Git contenant l'ensemble des fichiers du TP, avec vos captures d'écran de `kubectl get pods` (avant/après suppression d'un pod, avant/après scaling).

9 Questions de compréhension

1. Quelle est la différence entre une image Docker et un conteneur Docker ?
2. Pourquoi les données de la base sont-elles perdues si l'on supprime le volume, mais pas si l'on supprime seulement le conteneur ?
3. Un Deployment Kubernetes déclaré avec `replicas: 3` perd un pod. Que fait Kubernetes, et pourquoi parle-t-on d'approche *déclarative* ?
4. Quel est le rôle du Service par rapport aux pods, sachant que ces derniers sont recréés régulièrement avec une nouvelle adresse IP ?

5. Citez deux limites de Docker seul (sans Kubernetes) que vous avez observées ou apprises ce matin.

Pour aller plus loin (optionnel)

- Déployez également PostgreSQL sur Kubernetes (Deployment + Service + `PersistentVolumeClaim`) pour obtenir une stack 100 % Kubernetes.
- Externalisez les identifiants de connexion à la base dans un `Secret` Kubernetes plutôt que des variables en clair.
- Ajoutez des sondes `readinessProbe` et `livenessProbe` sur le conteneur `api` pointant vers `/health`.